Choreographer

System Description

Abstract

This document describes the Choreographer support system of the Eclipse Arrowhead 5.2.2 framework, as implemented in `ah5-Choreographer-java-spring` of the primary Java source tree (`Eclipse-Arrowhead-java`) and deployed with the standalone Compose definition of that folder.

Contents

1 Overview **3**
1.1 Significant Prior Art 4
1.2 How This System Is Meant to Be Used 4
1.3 System functionalities and properties 4
1.4 Important Delimitations 5

2 Services **6**
2.1 Produced services 6
2.2 Consumed services 6

3 Security **7**
3.1 Security model 7

4 References **7**

5 Revision History **9**
5.1 Amendments 9
5.2 Quality Assurance 9

1 Overview

This document describes the Choreographer system. The Choreographer executes workflow plans: for each step it selects an executor system, has DynamicServiceOrchestration orchestrate the required provider for the executor, starts the executor and follows the results.

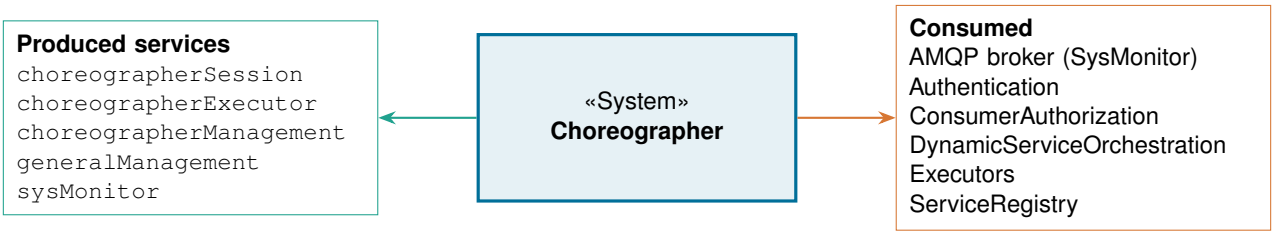


Figure 1: SysML-style block view of Choreographer with its produced and consumed services.

The rest of this document is organized as follows. In Section 1.1, we reference major prior art capabilities of the system. In Section 1.2, we describe the intended usage of the system. In Section 1.3, we describe fundamental properties provided by the system. In Section 1.4, we describe delimitations of capabilities of the system. In Section 2, we describe the abstract service functions consumed or produced by the system. In Section 3, we describe the security capabilities of the system.

1.1 Significant Prior Art

Successor of the Arrowhead 4.6 Choreographer core system.

1.2 How This System Is Meant to Be Used

Operators create plans; clients or operators start sessions; executors register and report step results. DSO pushes orchestration results to `POST /choreographer/orchestration-result/{correlationId}` (not a registered service).

1.3 System functionalities and properties

1.3.1 Functional properties of the system

- Plan validation (cycles, reachability, transitive reduction).
- Session engine with parallel independent steps.
- Executor selection and retry on `ERROR`; abort on `FATAL_ERROR`.
- Push orchestration for executors through DSO.

1.3.2 Configuration of system properties

The system is configured with Spring properties (`application.properties`, overridden by environment variables in the container deployment). The properties that shape its behaviour are:

Property	Default	Meaning
`server.port`	8457	HTTPS port.
`authentication.policy`	declared	Standalone deployment: `outsourced` with Authentication identity tokens and mutual TLS.
`management.policy` / `management.whitelist`	authorization	Access to the management services.
`enable.blacklist.filter` / `force.blacklist.filter`	false / true	Blacklist check of requesters.
`max.plan.iteration`	1000	Largest session quantity.
`orchestration.timeout`	30	Wait for a pushed orchestration result (s).
`session.worker.threads`	8	Engine worker threads.
`normalization.mode`	extended	Name normalization.
`sysmonitor.enabled`, `sysmonitor.broker.*`, `sysmonitor.ssl.*`	true, amqp-broker:5671, server.ssl stores	Native `sysMonitor` publisher of the common library.
`arrowhead.ssl.crl-file`	(unset)	CRL checked for every client certificate (standalone deployment: `/crl/ca.crl`).

Secrets (database, keystore and identity passwords, secret keys) have no usable defaults; the standalone deployment reads them from per-deployment Compose file secrets under `/run/secrets`.

The standalone deployment of this folder uses `config/application.properties` (copied into the image) with `server.port`=8454 (source default 8457), `authentication.policy=outsourced`, `service.interface.policy=CERT_AUTH`, `management.policy=sysop-only`, `service.authorization.enabled=true`, `mqtt.api.enabled=false`, `arrowhead.ssl.crl-file=/crl/ca.crl`; secrets are read from Compose file secrets under `/run/secrets` (`spring.config.import=configtree:/run/`

`secrets/`). `enable.blacklist.filter` is not enabled there, so incoming requests are not checked against the Blacklist; blacklisted providers are still removed by DynamicServiceOrchestration.

1.3.3 Data stored by the system

Plans, actions, steps, executors, sessions, session steps, worklog and log entries (database `ah_choreographer`).

1.3.4 Non functional properties

Steps run on a bounded worker pool; orchestration results time out after `orchestration.timeout`.

1.3.5 Stateful or stateless

Stateful; sessions active at restart are aborted.

1.4 Important Delimitations

No inter-cloud executor discovery; inter-cloud only through the orchestration flag.

2 Services

This section describes the produced and consumed services. Each produced service is specified by an SD document and one IDD document per interface.

2.1 Produced services

Service	Version	Interfaces	SD	IDD
`choreographerSession`	1.0.0	HTTP(S)	[1]	[2]
`choreographerExecutor`	1.0.0	HTTP(S)	[3]	[4]
`choreographerManagement`	1.0.0	HTTP(S)	[5]	[6]
`generalManagement`	1.0.0	HTTP(S)	[7]	[8]
`sysMonitor`	1.0.0	AMQP(S)	[9]	[10]

The HTTP interfaces are always offered and the AMQP `sysMonitor` interface with `sysmonitor.enabled=true`. The unregistered endpoints of the implementation are: `POST /choreographer/orchestration-result/{correlationId}`.

2.2 Consumed services

Provider	Service	Operations	Use	Condition
ServiceRegistry	systemDiscovery, serviceDiscovery	register, revoke, lookup	Registration of the system and its services; lookup of the bootstrap services.	always
Authentication	identity	identity-login, identity-verify	Own identity token; verification of requester tokens.	`authentication.policy=outsourced`
Consumer Authorization	authorization	verify	Management filter decisions.	`management.policy=authorization`
DynamicService Orchestration	service OrchestrationPush Management	subscribe, trigger	Orchestration for executors.	grant
Executors	executor interface	service-info, start, abort	Step execution.	sessions
AMQP broker (SysMonitor)	sysMonitor (publisher)	echo, power, status	Own observations.	`sysmonitor.enabled=true`

Except for the bootstrap services, which are looked up in the ServiceRegistry, the system reaches other systems' services through DynamicServiceOrchestration (`http.collector.mode=ORCHESTRATION`, results cached for 300 s); every such call needs a ConsumerAuthorization grant.

3 Security

The system can run in an insecure mode (`server.ssl.enabled=false`, plain HTTP, identity declared by the requester) for development, and in the Arrowhead secure mode with HTTPS and X.509 certificates following the Arrowhead certificate profile (`<System>.<Cloud>.<Organization>.arrowhead.eu`). Certificates not issued under the cloud CA are refused by the TLS handshake. The standalone deployment of this folder (`config/application.properties` and `compose.yaml`) runs the secure mode.

3.1 Security model

- **Protocols:** HTTPS (TLS 1.3 and 1.2) with mandatory client certificates; MQTT over TLS; AMQP over TLS for `sysMonitor`.
- **Data protection:** transport encryption by TLS; the database connection uses TLS with server verification in the standalone deployment. Secrets are kept out of the container environment (Compose file secrets).
- **System authentication:** by the authentication policy: `certificate`, `outsourced` (identity token verified with Authentication) or `declared` (insecure); the standalone deployment uses `outsourced`. An identity token is accepted only together with the client certificate of the system it was issued to (token-to-certificate binding), and every client certificate is checked against the cloud CA's CRL (fail closed).
- **Authorization of produced services:** management services are protected by the management filter (`management.policy`: `SYSOP_ONLY`, `WHITELIST` or `AUTHORIZATION`); with `service.authorization.enabled=true` ConsumerAuthorization decides each call; an optional blacklist filter refuses blacklisted requesters. Sysop identities bypass the management filter.
- The orchestration-result endpoint accepts only DynamicServiceOrchestration's certificate and pending random correlation ids.

For the Arrowhead certificate profile see github.com/eclipse-arrowhead/documentation.

4 References

[1] J. Delsing, "choreographerSession – Service Description (Choreographer)," Eclipse Arrowhead, SD 5.2.2, 2026, choreographer-v5.2.2-choreographerSession-SD.pdf.

[2] ——, "choreographerSession HTTP/TLS/JSON – Interface Design Description (Choreographer)," Eclipse Arrowhead, IDD 5.2.2, 2026, choreographer-v5.2.2-choreographerSession-HTTP-IDD.pdf.

[3] ——, "choreographerExecutor – Service Description (Choreographer)," Eclipse Arrowhead, SD 5.2.2, 2026, choreographer-v5.2.2-choreographerExecutor-SD.pdf.

[4] ——, "choreographerExecutor HTTP/TLS/JSON – Interface Design Description (Choreographer)," Eclipse Arrowhead, IDD 5.2.2, 2026, choreographer-v5.2.2-choreographerExecutor-HTTP-IDD.pdf.

[5] ——, "choreographerManagement – Service Description (Choreographer)," Eclipse Arrowhead, SD 5.2.2, 2026, choreographer-v5.2.2-choreographerManagement-SD.pdf.

[6] ——, "choreographerManagement HTTP/TLS/JSON – Interface Design Description (Choreographer)," Eclipse Arrowhead, IDD 5.2.2, 2026, choreographer-v5.2.2-choreographerManagement-HTTP-IDD.pdf.

[7] ——, "generalManagement – Service Description (Choreographer)," Eclipse Arrowhead, SD 5.2.2, 2026, choreographer-v5.2.2-generalManagement-SD.pdf.

[8] ——, "generalManagement HTTP/TLS/JSON – Interface Design Description (Choreographer)," Eclipse Arrowhead, IDD 5.2.2, 2026, choreographer-v5.2.2-generalManagement-HTTP-IDD.pdf.

[9] ——, “sysMonitor – Service Description (Choreographer),” Eclipse Arrowhead, SD 5.2.2, 2026, choreographer-v5.2.2-sysMonitor-SD.pdf.

[10] ——, “sysMonitor AMQP/TLS/JSON – Interface Design Description (Choreographer),” Eclipse Arrowhead, IDD 5.2.2, 2026, choreographer-v5.2.2-sysMonitor-AMQP-IDD.pdf.

5 Revision History

5.1 Amendments

No.	Date	Version	Subject of Amendments	Author
1	2026-10-22	5.2.2	Document created from the Arrowhead LaTeX templates for the 5.2.2 Java Spring implementation, including the local extensions of this workspace.	Jerker Delsing
2	2026-10-30	5.2.2	Checked against the primary Java source tree (Eclipse-Arrowhead-java): standalone deployment, configuration and security statements updated.	Claude Opus 5.5 (draft)

5.2 Quality Assurance

No.	Date	Version	Approved by
1		5.2.2	